

系统开发工具基础

第二周实验报告

乔海轩—25090012035

2026年8月31日

目录

1 总览	3
2 第 5 题控制一个可清理的后台任务	4
3 第 6 题语义重构与本地开发反馈	5
4 第 7 题用调试器定位归并排序缺陷	7
5 第 8 题先测量，再优化慢速词频程序	8
6 课后作业	10
6.1 q1: 捕获自定义信号 SIGUSR1	10
6.2 q2: 前后台切换练习	10
6.3 q3: 多任务控制	11
6.4 q4: 使用 wait 等待后台任务	11
6.5 q5: 重命名类名 Person→User	12
6.6 q6: 查找引用与跳转	12
6.7 q7: 使用 ruff 修复格式问题	12
6.8 q8: 调试递归斐波那契的边界条件	12
6.9 q9: 调试列表插入的越界	13
6.10 q10: 分析正则表达式重复编译的瓶颈	13
6.11 课后作业commit记录	14

1 总览

题目编号	描述	完成情况
5	控制一个可清理的后台任务	已完成
6	语义重构与本地开发反馈	已完成
7	用调试器定位归并排序缺陷	已完成
8	先测量，再优化慢速词频程序	已完成

Github 仓库地址: <https://github.com/Ptilotsis-o/system-dev-tools/>
Commit记录如图1所示。

```
42c9b45 (HEAD -> master, origin/master) feat(q08): optimize 'wordfreq.py'
c46a633 feat(q08): generate 'words.txt'
9a2077e feat(q08): files prepared
1028e33 fix(all): correct ignorance
1a8edd2 fix(all): correct .gitignore
b61968d feat(q07): add test file
c0ed133 feat(q07): fixed 'merge_sort.py'
e30d547 feat(q07): add 'breakpoint()' to 'merge_sort.py'
3db12d6 feat(q07): files prepared
fce6230 feat(q06): all passed
4c9471c feat(q06): add unused 'import os' to 'app.py'
163d49a feat(q06): replace 'total_price' to 'calculate_total'
6f3e40f feat(all): upgrade .gitignore
34a3b8c feat(week1): commit report
e18397f feat(week1/homework): final status
c996fab faet(q6): files prepared
b241c13 faet(q5): files prepared
```

图 1: Commit记录截图

2 第 5 题控制一个可清理的后台任务

题目要求

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9。

操作步骤与命令

```
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
Ctrl-Z
bg
jobs
pid=$(jobs -p)
kill $pid
jobs
cat cleanup.log
```

结果验证

- stdout.log 和 stderr.log 为空（符合预期）；
- cleanup.log 内容为 CLEAN_EXIT；
- 执行 kill 后，jobs 不再显示该任务，进程正常退出；
- 未使用 kill -9，证明进程捕获了 SIGTERM 并执行了清理。

```

qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week2/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week2/q05$ jobs
[1]+  运行中      ./worker.sh > stdout.log 2> stderr.log &
qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week2/q05$ pid=$(jobs -p %1)
qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week2/q05$ kill $pid
qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week2/q05$ jobs
[1]+  已完成      ./worker.sh > stdout.log 2> stderr.log
qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week2/q05$ cat cleanup.log
CLEAN_EXIT

```

图 2: 部分指令截图

3 第 6 题语义重构与本地开发反馈

题目要求

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 `total_price` 改为 `calculate_total`，保证定义和所有代码引用同步改变。
- 在 `app.py` 中临时加入未使用的 `import os`，用 `ruff check` 发现并删除；最后运行 `ruff check` 和 `pytest` 并保证通过。

操作步骤与命令

```

source venv/bin/activate
vim app.py
# 完成 gd、gf、重命名等操作
vim app.py # 临时加入 import os
ruff check
vim app.py # 删除 import os
ruff check
pytest

```

结果验证

- `gd` 能够跳转到定义 `total_price` 定义，`gf` 能够跳转到 `total_price` 引用位置；

- \rn 将 total_price 重命名为 calculate_total;
- ruff 不报错, pytest 通过。

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q06$ ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
| 1 | / import os
| 2 | from math_utils import calculate_total
| 3 | print(calculate_total(12.5, 4))
|
help: Organize imports
|
| 1 | import os
| 2 +
| 3 | from math_utils import calculate_total
| 4 +
| 5 | print(calculate_total(12.5, 4))
|

F401 [*] 'os' imported but unused
--> app.py:1:8
|
| 1 | import os
|   | ^
| 2 | from math_utils import calculate_total
| 3 | print(calculate_total(12.5, 4))
|
help: Remove unused import: 'os'
|
| - import os
| 1 | from math_utils import calculate_total
|
Found 2 errors.
[*] 2 fixable with the '--fix' option.
```

图 3: ruff报错时截图

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q06$ ruff check
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
| 1 | / import os
| 2 | from math_utils import calculate_total
| 3 | print(calculate_total(12.5, 4))
|
help: Organize imports
|
| 1 | import os
| 2 +
| 3 | from math_utils import calculate_total
| 4 +
| 5 | print(calculate_total(12.5, 4))
|

F401 [*] 'os' imported but unused
--> app.py:1:8
|
| 1 | import os
|   | ^
| 2 | from math_utils import calculate_total
| 3 | print(calculate_total(12.5, 4))
|
help: Remove unused import: 'os'
|
| - import os
| 1 | from math_utils import calculate_total
|

Found 2 errors.
[*] 2 fixable with the '--fix' option.
```

图 4: ruff和pytest检查截图

4 第 7 题用调试器定位归并排序缺陷

题目要求

- 先运行给定输入，确认结果错误或出现异常。
- 使用 pdb、IDE debugger 或等价调试器，在 merge 中设置断点并观察 i、j、left[i]、right[j]。
- 完成最小修复，不得用 sorted 替换归并排序。
- 使用 pytest 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

操作步骤与命令

```
python merge_sort.py
vim merge_sort.py
python merge_sort.py
# 调试指令，此处略
vim merge_sort.py
vim test_merge_sort.py
pytest
```

结果验证

- 初次运行merge_sort.py，结果错误。
- 为merge_sort.py 添加断点，使用 pdb，观察 i、j、left[i]、right[j]。
- 修复merge_sort.py 中的错误，确保归并排序正确。
- 编写测试用例，运行pytest，确认测试通过。



```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q075 python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 5: 初次运行时的错误结果

```

(Pdb) n
> /home/qiao-haixuan/system-dev-tools/week2/q07/merge_sort.py(9)merge()
-> result.append(right[i]); j += 1 # 此处存在缺陷
(Pdb) p left
[1, 3]
(Pdb) p right
[1, 4]
(Pdb) p i
1
(Pdb) p j
9
(Pdb) p right[i]
4
(Pdb) p right[j]
1
(Pdb)

```

图 6: pdb发现逻辑错误时的截图

图 7: pytest输出截图

5 第 8 题先测量，再优化慢速词频程序

题目要求

- 生成 words.txt 文件，使用 time 运行原程序 2 次并记录耗时。
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置。
- 在不改变最终 count 的前提下优化程序；不得写死 1000。
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

操作步骤与命令

```

python generate_words.py
time python word_freq.py
time python word_freq.py
cProfile -s cumulative word_freq.py
vim word_freq.py
time python word_freq.py

```



```
time python word_freq.py
```

结果验证

- 生成了 words.txt 文件。
- 运行原程序和优化后的程序各 2 次，记录耗时，cProfile -s cumulative 输出主要耗时位置。
- 优化前中位数为 0.1685s，优化后中位数为 0.0315s，加速比为 81.3%。

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q08$ time python wordfreq.py
count= 1000

real    0m0.172s
user    0m0.158s
sys     0m0.005s
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q08$ time python wordfreq.py
count= 1000

real    0m0.165s
user    0m0.159s
sys     0m0.007s
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q08$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.143 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cuntime  percall  filename:lineno(function)
1      0.000    0.000    0.143    0.143  {built-in method builtins.exec}
1      0.141    0.141    0.143    0.143  wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001  {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000  {method 'read' of 'io.TextIOWrapper' objects}
1000    0.000    0.000    0.000    0.000  {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000  <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000  {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000  {built-in method builtins.print}
1      0.000    0.000    0.000    0.000  {built-in method _io.open}
1      0.000    0.000    0.000    0.000  {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000  <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000  {built-in method builtins.len}
1      0.000    0.000    0.000    0.000  <frozen codecs>:263(__init__)
```

图 8: 优化前 time 和 cProfile 输出截图

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q08$ time python wordfreq.py
count= 1000

real    0m0.030s
user    0m0.014s
sys     0m0.008s
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/q08$ time python wordfreq.py
count= 1000

real    0m0.033s
user    0m0.014s
sys     0m0.007s
```

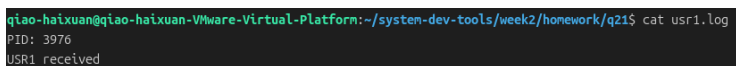
图 9: 优化后 time 截图

6 课后作业

6.1 q1: 捕获自定义信号 SIGUSR1

目标: 创建 `sig_demo.sh`, 使用 `trap` 捕获 `SIGUSR1`, 收到信号时向 `usr1.log` 写入 "USR1 received"。在后台启动脚本, 通过 `kill -USR1 <pid>` 发送信号, 验证日志内容。

```
chmod +x sig_demo.sh
./sig_demo.sh &
pid=$!
kill -USR1 $pid
cat usr1.log
kill $pid
```



A terminal window showing the execution of the script. The prompt is `qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q21$`. The user runs `cat usr1.log`, and the output is `PID: 3976` followed by `USR1 received` on the next line.

图 10: `kill -USR1 $pid` 发送信号, 验证日志内容

6.2 q2: 前后台切换练习

目标: 启动 `ping 127.0.0.1 > ping.log`, 使用 `Ctrl-Z` 挂起, 再用 `bg` 放到后台, 然后用 `fg` 调回前台, 最后用 `Ctrl-C` 终止。记录每一步的 `jobs` 输出。

```
ping 127.0.0.1 > ping.log &
Ctrl-Z
jobs
bg
jobs
fg
ctrl-c
```

```

qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q22$ ping 127.0.0.1 > ping.log
^Z
[1]+ 已停止                  ping 127.0.0.1 > ping.log
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q22$ jobs
[1]+ 已停止                  ping 127.0.0.1 > ping.log
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q22$ bg
[1]+ ping 127.0.0.1 > ping.log &
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q22$ jobs
[1]+ 运行中                  ping 127.0.0.1 > ping.log &
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q22$ fg
ping 127.0.0.1 > ping.log
^Cqiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q22$

```

图 11: ping 127.0.0.1 & ping.log & 挂起、放到后台、调回前台、终止，记录 jobs 输出

6.3 q3: 多任务控制

目标：同时启动两个脚本 w1.sh（无限输出 A）和 w2.sh（无限输出 B），将 stdout 分别重定向到 a.log 和 b.log。用 jobs 查看状态，并用 kill %1 %2 同时终止两个任务。

```

chmod +x w1.sh
chmod +x w2.sh
./w1.sh > a.log &
./w2.sh > b.log &
jobs
kill %1 %2

```

```

qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q23$ ./w1.sh > a.log 2>&1 &
./w2.sh > b.log 2>&1 &
[1] 5851
[2] 5852
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q23$ jobs
[1]- 运行中                  ./w1.sh > a.log 2>&1 &
[2]+ 运行中                  ./w2.sh > b.log 2>&1 &
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q23$ kill %1 %2
[1]- 已终止                  ./w1.sh > a.log 2>&1
[2]+ 已终止                  ./w2.sh > b.log 2>&1
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q23$

```

图 12: jobs 输出截图，终止两个任务后为空

6.4 q4: 使用 wait 等待后台任务

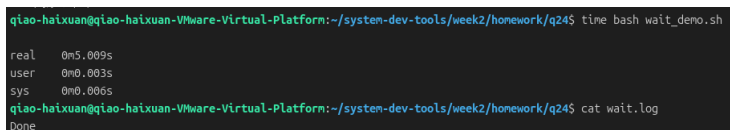
目标：编写脚本，后台启动 sleep 5，使用 wait 等待其结束，结束后输出 "Done" 到 wait.log。用 time 测量整体耗时，确认等待生效。

```

time bash wait_demo.sh

```

```
cat wait.log
```



```
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q24$ time bash wait_demo.sh
real    0m5.009s
user    0m0.003s
sys     0m0.006s
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week2/homework/q24$ cat wait.log
Done
```

图 13: wait.log 输出截图

6.5 q5: 重命名类名 Person→User

目标: 创建 `person.py` 定义类 `Person`, 在 `main.py` 中实例化并调用。使用语言服务器的“重命名符号”功能将 `Person` 改为 `User`, 确保所有引用同步更新, 并运行验证。将光标移至 `person.py` 中的 `Person` 类名, 通过 `\ rn` 重命名, 修改为 `User`。

6.6 q6: 查找引用与跳转

目标: 在 `const.py` 中定义常量 `PI = 3.14159`, 并在 `area.py`、`volume.py` 中导入使用。使用“查找所有引用”列出所有使用 `PI` 的位置, 并用“跳转到定义”确认来源。将光标移至 `const.py` 中的 `PI` 定义, 通过 `gr` 查找所有引用。

6.7 q7: 使用 ruff 修复格式问题

目标: 创建格式混乱的 `messy.py` (含多余空格、缺少空行、行尾空白等)。运行 `ruff check --fix messy.py` 自动修复, 再运行 `ruff format` 统一格式, 确保无报错。

```
ruff check --fix messy.py
ruff format messy.py
ruff check messy.py
```

6.8 q8: 调试递归斐波那契的边界条件

目标: 提供有缺陷的 `fib` 函数 (当 `n=0` 时返回 1)。使用调试器单步跟踪 `fib(0)` 的调用过程, 定位错误并修复, 然后编写 `pytest` 测试验证

```
vim fib_bug.py
pytest test_fib.py
```

图 14: fib_bug.py 调试截图

图 15: pytest test_fib.py 输出截图

目标：供 `safe_insert` 函数，当索引越界时抛出 `IndexError`。调用 `safe_insert([1,2,3], 5, 99)` 触发异常，使用调试器观察 `idx` 与 `len(arr)`，修复逻辑（如越界时追加到末尾），并添加测试。

目标: 创建 `regex_slow.py`, 在循环内部反复调用 `re.compile`。使用 `cProfile` 定位 `re.compile` 热点, 将编译提到循环外进行预编译优化, 测量前后耗时并计算加速比。

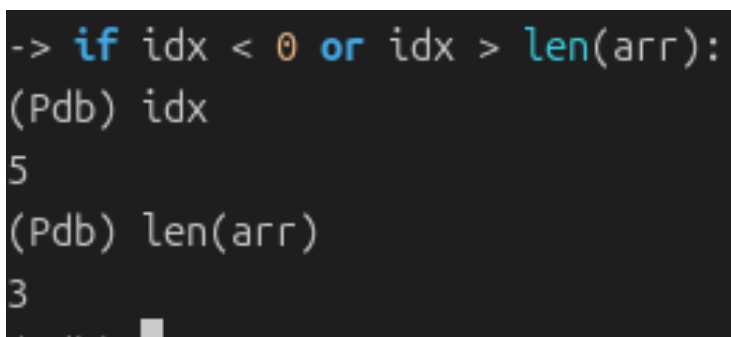


图 16: safe_insert.py 调试截图

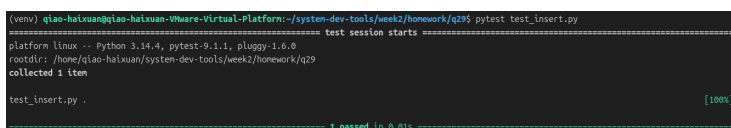


图 17: pytest test_safe_insert.py 输出截图

```
time python regex_slow.py
time python regex_slow.py
python -m cProfile -s cumulative regex_slow.py
time python regex_fast.py
time python regex_fast.py
```

优化前时间中位数为0.020秒，优化后中位数为0.016秒。

6.11 课后作业commit记录

7 解题感悟

本次实验报告涵盖了系统开发工具基础的进程管理、代码重构、调试与性能优化等方面。通过实际操作，我掌握了信号捕获、前后台任务切换、语言服务器重命名、pdb调试以及使用 time 和 cProfile 定位瓶颈。课后作业进一步巩固了这些技能，尤其是在处理自定义信号、正则预编译优化等问题时，提升了分析与解决问题的能力。总的来说，这次实验让我对命令行环境和开发工具有了更深入的理解，为后续工程实践打下了基础。

```
* 673c613 (HEAD -> master, origin/master, origin/HEAD) feat(q210): finished
* c596c99 feat(q29): finished
* 68fa53b feat(q29): file prepared
* d99996e feat(q28): finished
* 21aa469 feat(q28): add breakpoint()
* e6d4ea4 feat(q28): file prepared
* 1d04ff3 feat(q27): finished
* 502b678 feat(q27): file prepared
* e09264c fix(q26): files prepared
* 17762e6 feat(q26): files prepared
* b8b4108 feat(q25): finished
* c42a507 feat(q25): files prepared
* d38ad16 feat(q24): finished
* 2451bb1 feat(q24): file prepared
* 13c39d4 feat(q23): finished
* b8cc42a feat(q23): files prepared
* 2e81098 feat(q22): finished
* 2ab47d8 feat(q21): finished
* f49ff02 feat(q21): file prepared
```

图 18: 课后作业commit记录